This Week (Week 1) — Nano-Rare GT Framework

Monday: Project Setup + Architecture

- ☐ `cd ~/nano-rare-gt && mkdir src/nanogt tests data docs`
- ☐ Create `pyproject.toml` with dependencies: `typer`, `requests`, `pydantic`, `jinja2`, `pytest`, `requests-cache`, `pandas`
- ☐ Initialize git repo (done), set up `pre-commit` with `black`, `ruff`, `mypy`
- ☐ Write `docs/ADR-001-architecture.md` — SQLite v1, Pydantic v2 models, Typer CLI
- ☐ Define core Pydantic models in `src/nanogt/models.py`:
  - `Disease` (orphanet_id, name, prevalence, morbidity_flag)
  - `Gene` (symbol, omim_id, uniprot_id, cds_length, aa_length)
  - `Protein` (uniprot_id, afdb_id, sequence, domains[])
  - `Vector` (serotype, cargo_limit, tissue_tropism[])
  - `Match` (disease, gene, vector, surrogate_program, scores{})
  - `ScoreBreakdown` (structural, size, tissue, roa, promoter, localization, immuno, toxicity, cai)
- ☐ Define SQLite schema in `src/nanogt/schema.sql`
- ☐ **ROGDI seed:** Create `data/rogdi_test_fixture.json` with source-audited facts:
  - gene=ROGDI, aliases=[KIAA0267, FLJ22386, RAV2], orphanet=ORPHA:1946, omim_phenotype=226750 omim_gene=614574, uniprot=Q9GZN7, aa=287, cds_bp~861/864 including stop codon
  - domains/structure=[ROGDI/RAVE2-like scaffold, InterPro IPR028241, Pfam PF10259, PDB 5XQH/5XQI]
  - biology=[intracellular/non-secreted, Rabconnectin-3 / V-ATPase-linked function, cell-autonomous rescue likely]
  - cell_types=[CNS neurons/presynaptic compartments, ameloblast-lineage enamel tissue, possible renal tubules]
  - phenotype=[amelogenesis imperfecta, early-onset epilepsy, severe developmental delay/intellectual disability/regression, spasticity, hypohidrosis, nephrocalcinosis reported in some cases]

Tuesday: Disease Discovery Module

- ☐ Implement `src/nanogt/disease.py`:
  - `query_orphanet(rarity_threshold="nano-rare")` → list[Disease]
  - `filter_no_active_trials(diseases)` → cross-ref ClinicalTrials.gov
  - `filter_high_morbidity(diseases)` → boost if Orphanet disability weight > threshold
  - `link_to_omim(disease)` → resolve OMIM gene entry
- ☐ Add `requests-cache` to all HTTP clients (7-day TTL default)
- ☐ Unit tests in `tests/test_disease.py` with mocked Orphanet/OMIM responses
- ☐ **ROGDI test:** Verify ORPHA:1946 resolves to Kohlschütter-Tönz syndrome / amelocerebrohypohidrotic syndrome with OMIM phenotype 226750 and OMIM gene 614574

Wednesday: Protein + Structure Linkage

- ☐ Implement `src/nanogt/homology.py`:
  - `fetch_uniprot(uniprot_id)` → sequence, domains, GO terms
  - `fetch_alphafold(uniprot_id)` → PDB file URL or structural summary
  - `calculate_domain_similarity(protein_a, protein_b)` → cosine on domain vectors
  - `sequence_identity(a, b)` → global + local alignment scores
- ☐ Store in SQLite via `src/nanogt/db.py` (Connection manager, insert/update/query)
- ☐ Unit tests in `tests/test_homology.py`

- ☐ **ROGDI validation:** Verify gene-symbol search for ROGDI returns UniProt Q9GZN7 → 287 aa, Protein rogdi homolog, PDB 5XQH/5XQI, InterPro IPR028241, Pfam PF10259; do not use obsolete reductase-style scaffold annotations

Thursday: Vector Sizing + Serotype Check

- ☐ Implement `src/nanogt/vector.py`:
  - `is_aav_compatible(cds_length_bp)` → bool (gate: <= 4700)
  - `aav_serotypes` → static dictionary: name → tissue list, cargo limit, clinical precedent count
  - `match_serotype_to_tissue(serotype, target_tissues)` → compatibility score
  - `precedent_count(serotype, promoter)` → platform depth metric
- ☐ Build `data/serotype_map.json` from literature
- ☐ Unit tests in `tests/test_vector.py`
- ☐ **ROGDI sizing:** Confirm ~861 bp amino-acid coding region (~864 bp including stop codon) fits comfortably within AAV. Evaluate AAV9 and AAVrh.10 for CNS-first delivery; treat evolved CNS capsids and any ameloblast-targeting route as research-stage/uncertain rather than proven.

Friday: CLI Integration + v0.1 End-to-End

- ☐ Implement `src/nanogt/cli.py` (Typer):
  - `nanogt match --disease ORPHA:1946 --output report.md` # ROGDI/KTS as primary example
  - `nanogt init` → create ~/.nanogt/ directory + SQLite DB
  - `nanogt status` → check DB health, API connectivity
- ☐ Implement `src/nanogt/report.py`:
  - Markdown output (Jinja2 template)
  - JSONL for downstream analysis
  - ROGDI-specific sections: cell-type targeting, delivery routes, therapeutic window
- ☐ Run single end-to-end test: `nanogt match --disease ORPHA:1946 --output rogdi_report.md`
  - Must complete in < 2 min
  - Must produce report with plausible surrogate match for KTS
  - Must include: gene size, AAV compatibility, CNS + dental targeting assessment
- ☐ Supervisor review: Deliver ROGDI report as proof-of-concept
- ☐ Fix all blockers, tag `v0.1-PoC`
- ☐ Run `pytest` — all tests green
- ☐ Run `mypy src/` — clean
- ☐ Review: `/review` then `/ship`